

Smart Parking Management System for HCMUT (IoT-SPMS)

Submission #1: Requirements

Course: Software Engineering (SE252) Project: Smart Parking System for University Campus, Ho Chi Minh City University of Technology (HCMUT)

Contents

1. Context and overview
2. Stakeholders
3. Objectives and scope
4. Actors
5. Functional requirements
6. Use-case diagram
7. Non-functional requirements
8. Assumptions and open questions
9. Glossary

1. Context and overview

HCMUT has two campuses (Lý Thường Kiệt in District 10 and Dĩ An in Bình Dương) and a large number of people who ride in every day: students, graduate students, lecturers, staff, and visitors. Parking is already managed with RFID cards that were rolled out around 2019, mostly for motorbikes and bicycles.

The current setup has a few problems we noticed and that the brief also mentions:

- Long queues at the gates in the morning, because almost everyone arrives in the short window before class starts.
- Cards get lost or copied, and a card is not really tied to a specific vehicle.
- Fees are still handled with cash and checked by the guard by hand.
- There is no easy way to see how many spaces are free before you drive in.

The system we are proposing, the IoT-based Smart Parking Management System (IoT-SPMS), tries to fix these. It logs each entry and exit against the rider's HCMUT account through the university SSO, uses sensors to know which slots are taken, shows how full each area is on signs at the gate, works out the fee and sends the payment request to BKPay, and gives the operators and admins a live view plus a record of everything for auditing.

One thing that shaped almost every decision in this project: on a Vietnamese campus the parking is mostly motorbikes, not cars. Most students own a motorbike and very few own a car, so the main lane

has to move a lot of motorbikes quickly, not park one car per bay like a Western car park. A slot is roughly 1m x 2m and there are a lot of them.

Because of that, a normal barrier gate does not work well here. A barrier takes about 1.5 seconds to open and close, so one lane can only pass around 40 vehicles a minute, and during the morning rush there are far more people than that arriving. So for the motorbike lane we assume a barrier-free flow: a camera reads the plate and the rider taps the card while moving, and the plate is checked again on the way out. Barriers are only used for the smaller car lot.

This also gives us a way to handle bike theft, which is a real concern here. When someone enters, we store the card, the plate read by the camera, a photo, and the time. When they leave, we read the plate again for the same card. If the plate does not match the one from entry, we treat it as a possible theft and raise an alarm. This "plate tied to card" idea is one of the more important parts of the design and it shows up in the requirements below.

2. Stakeholders

The people and groups who care about this system:

- Members who ride in (students, lecturers, staff). They want to get in and out fast, pay a fair fee, see free spaces, and manage their own account and vehicles. Students get a discount when they show a valid ID.
- Visitors. People from outside with no HCMUT account. They need a simple way to get in (a ticket the operator gives them) and to pay when they leave.
- Parking operators. They watch the live board, deal with problems (a stuck barrier, a lost ticket, an alarm), take cash, and look up a vehicle by its plate. Each operator is tied to the lot they work at.
- System administrators. They set the fees, manage users and the blacklist, configure zones/slots/sensors, look at the dashboards and reports, and can see the audit log.
- The university finance office. They care that the fee is right and that the money collected matches what the bank reports.
- Facilities and security staff. They care about barrier safety and theft.
- HCMUT IT. They own the SSO and the student data, so any integration goes through them.
- University management, who are paying for it and want less congestion and a more modern experience.

3. Objectives and scope

What we want the system to do:

- O1. Record entry and exit automatically, for members through their SSO account and for visitors through a temporary ticket, without needing the login server to be reachable at the exact moment they pass the gate.
- O2. Keep a close-to-live count of free spaces that still works when a sensor or the network drops out.

- O3. Show how full each area is on the signs and point drivers toward areas that still have room.
- O4. Add up a member's parking over a billing period, work out the fee, and send the payment request to BKPay. Visitors pay by cash or QR.
- O5. Enforce roles (normal user, operator, admin) and keep a record of parking and money activity.
- O6. Keep working under the real conditions here: crowds at peak times, a network that is not always up, and very different kinds of users.

In scope: entry/exit, slot tracking, signage, the availability view, sessions and billing, payment through BKPay or cash, visitor tickets, SSO login, reading student data from DATACORE, roles, admin configuration, dashboards and reports, and the audit log.

Out of scope: buying and installing the actual sensors and barriers; building HCMUT_SSO, DATACORE, or BKPay themselves (we treat them as systems that already exist); towing wrongly parked vehicles; anything outside the campus.

A few scope choices we made for the demo:

- Reservation (letting someone book a slot ahead) is optional. We list the requirements but treat it as a bonus.
- BKPay is designed in but stubbed in the prototype. The real BKPay is a web-only portal tied to one bank and it does not expose an API we could call for parking, so we mock it and note a real integration as future work.
- The prototype shows one lot, but the data model is built so more lots and more gates can be added later.

4. Actors

We separate two things that are easy to mix up. One is the role in the app (what screens and actions you get), which is a permissions question. The other, only for end users, is whether you are a student, lecturer, staff, or visitor, which mainly changes the price you pay. We keep that as an attribute on the user, not as a separate login role.

People who use the system directly:

- Member driver (role: end user). A student, lecturer, or staff member with an HCMUT account. Logs in, sees free spaces, enters and leaves, gets billed, tops up a prepaid balance, and manages their own vehicles and history.
- Visitor driver (role: end user, type visitor). Someone from outside with no account. The operator opens a plate-only session for them and they pay by cash or QR at the exit.
- Parking operator (role: operator). Watches the board, handles alarms and problems, issues visitor tickets, looks up vehicles, takes cash. Tied to the lot they work at.
- System administrator (role: admin). Sets fees, manages users and roles, configures zones/slots/sensors, sees dashboards, reports, and the audit log.

Outside systems that also take part in the use cases:

- Slot sensor, which reports a slot as taken or free.
- Barrier controller, for the car lane.
- Card reader, which reads the member card.
- Camera (ANPR), which reads the plate.
- The signs at the gate.
- HCMUT_SSO, the university login.
- HCMUT_DATACORE, the read-only source of student and vehicle data.
- BKPay, the payment gateway.

5. Functional requirements

We grouped the requirements into seven modules. Each one is written as "the system shall ..." and has an ID so we can point to it later. Priority is M (must have), S (should have), or C (could have).

A. Entry and access control

ID	Pri	Requirement
FR-ENT-01	M	The system shall detect a vehicle at the entry lane and start reading its credential.
FR-ENT-02	M	The system shall read the member's card (or a QR code) and read the plate from the camera at entry.
FR-ENT-03	M	The system shall check the credential against active membership, the blacklist, and whether the lot is full before letting the vehicle in.
FR-ENT-04	M	When a valid member enters, the system shall open a session recording the time, lane, card, plate, and entry photo.
FR-ENT-05	M	When there is no member card, the system shall run the visitor flow: issue a QR ticket and open a plate-only session that does not depend on the SSO.
FR-ENT-06	M	The system shall refuse entry and show a reason when the lot is full, the credential is invalid, or the vehicle is on the blacklist.
FR-ENT-07	S	For the car lane, the system shall open the barrier when entry is allowed and record that it opened.
FR-ENT-08	M	The gate shall keep working during a network outage by deciding from a local cache and sending the queued events once it reconnects.

B. Slot occupancy

ID	Pri	Requirement
FR-OCC-01	M	The system shall receive the taken/free changes reported by the slot sensors.
FR-OCC-02	M	The system shall keep a current state for each slot: free, taken, reserved, or out of service.
FR-OCC-03	M	The system shall keep a count of free spaces per area and per lot.
FR-OCC-04	M	The system shall compare the sensor counts against the entry/exit counts now and then and flag differences.
FR-OCC-05	M	If a sensor has not reported for too long, the system shall mark that slot as "unknown" and shall not count it as free.
FR-OCC-06	S	The system shall flag a sensor that looks stuck or broken and raise a maintenance alert.
FR-OCC-07	C	The system shall reset the count at night when the lot empties, to correct drift.

C. Signage and guidance

ID	Pri	Requirement
FR-SIG-01	M	The system shall work out a state for each area from the free count and thresholds the admin sets.
FR-SIG-02	M	The system shall show the lot state on the entrance sign (spaces left, nearly full, or full).
FR-SIG-03	S	The system shall point drivers toward an area that still has room, and toward the nearest non-full area when their area is full.
FR-SIG-04	M	The admin shall be able to set the percentages that decide each sign state. We use green below 75%, yellow up to 90%, then full.
FR-SIG-05	M	The system shall show the current availability in a web/mobile view with the time it was last updated.

D. Exit and billing

ID	Pri	Requirement
FR-EXT-01	M	The system shall read the card (or ticket) and read the plate again at exit.
FR-EXT-02	M	The system shall find the open session for that credential.
FR-EXT-03	M	The system shall work out the fee from how long the vehicle stayed and the price rule for its type and the user's category, using the rounding and daily cap that are set.
FR-EXT-04	M	The system shall compare the exit plate with the entry plate and raise a theft alarm if they do not match, holding the session for the operator.
FR-EXT-05	M	The system shall take payment from the prepaid balance, BKPay, or cash, and close the session once it is paid.
FR-EXT-06	M	The system shall save the amount charged on the record when the session closes, so a later price change does not change old bills.
FR-EXT-07	S	For the car lane, the system shall open the exit barrier only after payment is done or a valid pass is confirmed.
FR-EXT-08	M	The system shall add up a member's charges over the billing period and send a payment request to BKPay for the total.

E. Reservation (optional)

ID	Pri	Requirement
FR-RES-01	C	The system shall let a member book a slot for a time window.
FR-RES-02	C	The system shall hold a booked slot and release it if the member does not show up in time.
FR-RES-03	C	The system shall turn a booking into an open session when the member enters in the window.

F. Administration and reports

ID	Pri	Requirement
FR-ADM-01	M	The admin shall be able to create and update price rules with a start and end date.
FR-ADM-02	M	The admin shall be able to manage users, cards, vehicles, the blacklist, and role assignments.
FR-ADM-03	M	The admin shall be able to configure zones, slots, sensors, and signs.
FR-ADM-04	M	The system shall show dashboards for occupancy, revenue, and busy hours.
FR-ADM-05	S	The system shall let the admin export parking and financial reports for a period.
FR-ADM-06	S	The system shall check the payments it recorded against the report the bank sends and flag any that do not match.

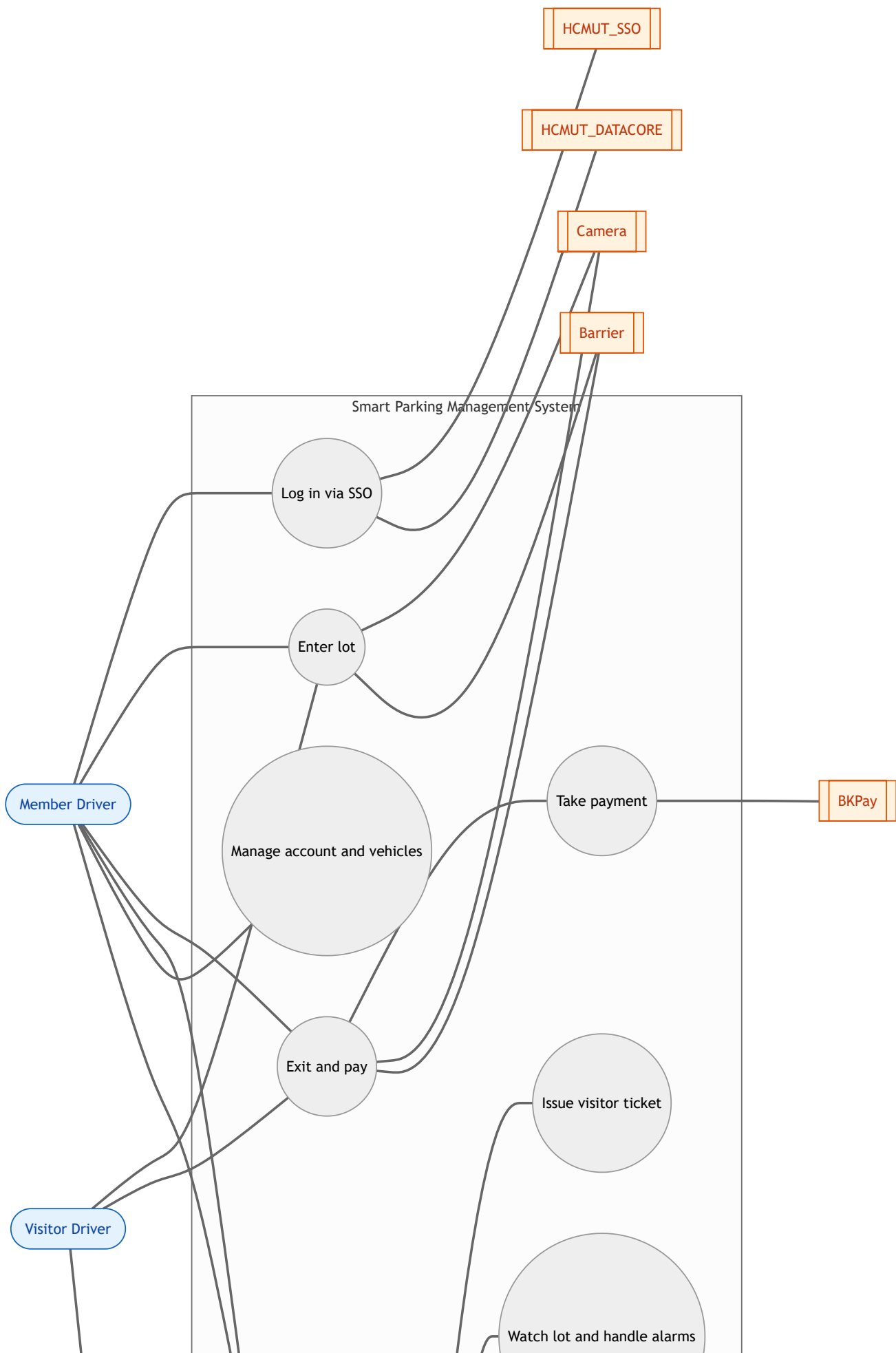
G. Monitoring and audit

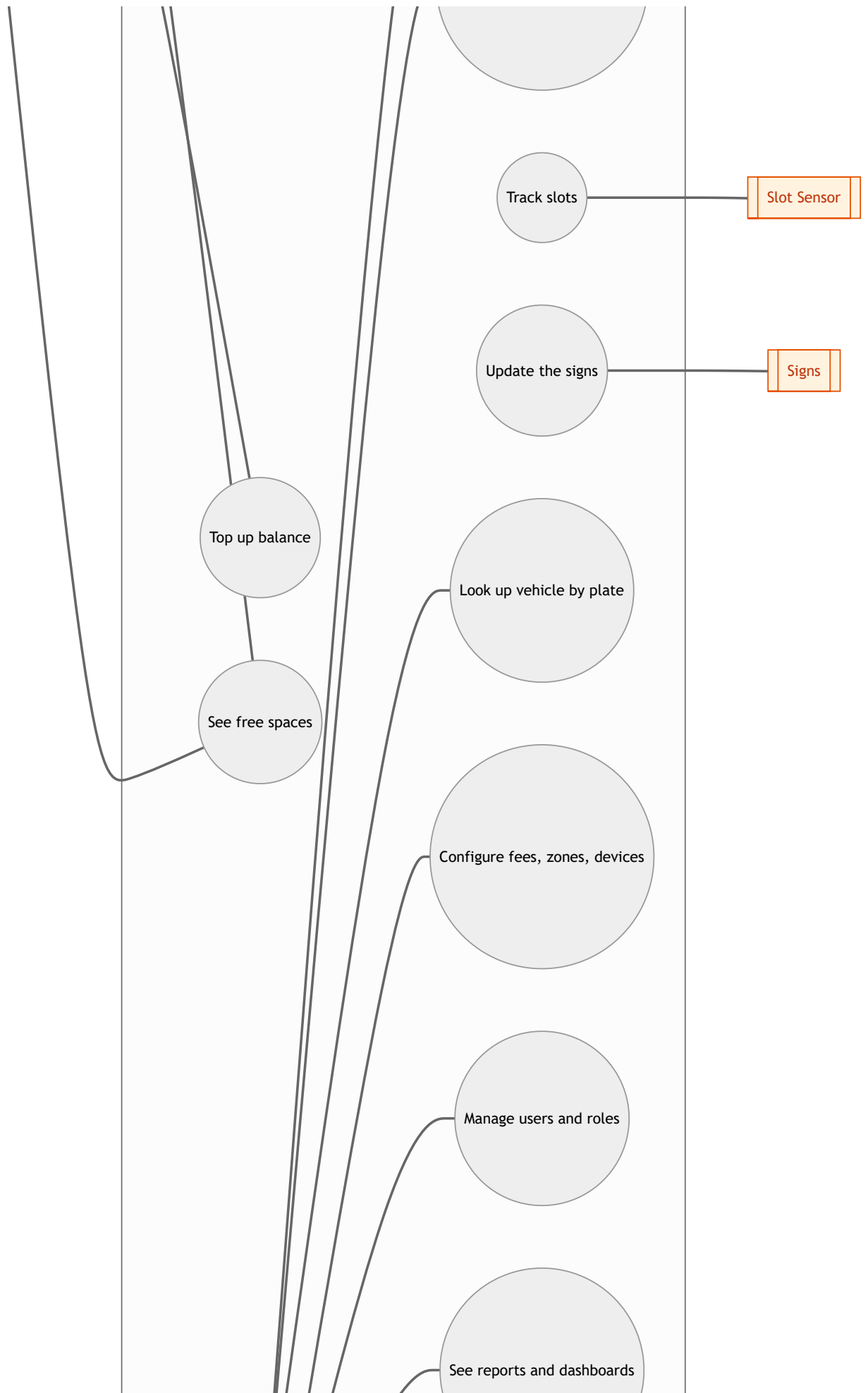
ID	Pri	Requirement
FR-AUD-01	M	The system shall show operators a live board of occupancy, open sessions, and open alarms.
FR-AUD-02	M	An operator shall be able to look up an open session or vehicle by plate.
FR-AUD-03	M	The system shall raise alarms for hardware faults, a forced barrier, a plate mismatch, or the same vehicle entering twice, and let the operator acknowledge them.
FR-AUD-04	M	The system shall keep a record of every entry, exit, payment, and admin action, with who did it and when.

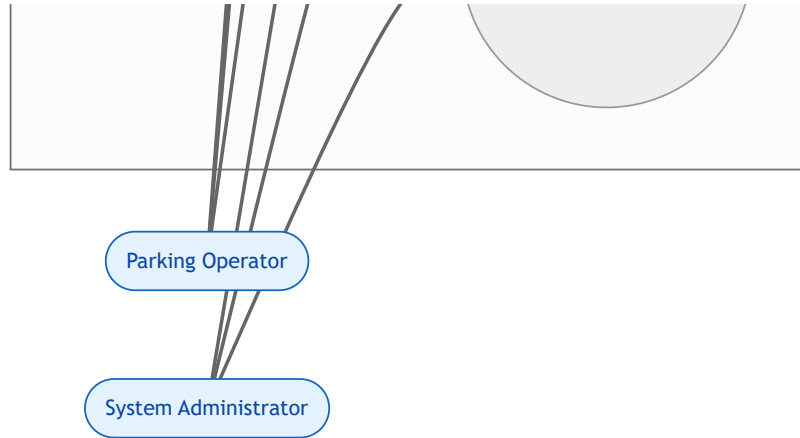
Most of the interesting edge cases live in these modules: a full lot, a broken sensor, the network being down, a lost ticket, a cloned card with the wrong plate, the same vehicle entering twice, and a payment callback that never arrives.

6. Use-case diagram

Members and visitors are both end users; a visitor session is opened by the operator. The outside systems are on the right.







Some relationships between the use cases: "Enter lot" includes checking the slot count and reading the plate. "Exit and pay" includes "Take payment", and the theft alarm on a plate mismatch is an extension of it. "Enter lot" extends into "Issue visitor ticket" when there is no member card. "Log in via SSO" is used by every member action.

7. Non-functional requirements

We grouped these by the quality types in ISO/IEC 25010. The numbers are targets we would test against, not measurements we have already taken. For this project the ones that matter most are staying up when the network drops, coping with a broken sensor, keeping the gate and signs fast, and keeping card and payment data safe.

ID	Quality	Requirement	Target	Check
NFR-PERF-01	Performance	Time from a valid card read to the barrier opening.	About 2 seconds, at most 3.	Timing test
NFR-PERF-02	Performance	How fast the availability view and signs show a change.	Within a few seconds (about 5).	End-to-end test
NFR-PERF-03	Performance	Camera plate reading.	Under 2 seconds, and correct most of the time.	Test on sample images
NFR-CAP-01	Capacity	Handling many gate events and open sessions at once during peak.	Should hold up to a few thousand open sessions without slowing down.	Load test
NFR-REL-01	Reliability	Uptime.	High, aiming for 99.5% a month.	Monitoring
NFR-REL-02	Reliability	Entry/exit keeps working when the backend is unreachable and syncs when it comes back.	Local decision, sync within about a minute.	Fault-injection test
NFR-REL-03	Reliability	One broken sensor must not break the free count; the slot goes to "unknown".	Alerted quickly, count never wrong.	Fault-injection test
NFR-REL-04	Reliability	Recover the last state after a power cut or crash, keeping open sessions.	Back up within a couple of minutes.	Recovery test
NFR-SEC-01	Security	Card and payment data encrypted when stored and sent; not written in plain logs.	Encrypted storage, HTTPS.	Review
NFR-SEC-02	Security	Every action is allowed only for the right role, and a user can only see their own session.	No unauthorized action passes the tests.	Access test
NFR-SEC-03	Security	Entry, exit, payment, and admin actions are all logged and kept.	Full coverage, kept for a long period.	Log review
NFR-USE-01	Usability	The entrance sign is readable from a distance and an operator can find a vehicle quickly.	Readable across the lane, find-by-plate in well under a minute.	Usability test
NFR-SAFE-01	Safety	The barrier must not close on a person or vehicle.	No unsafe closing.	Safety test
NFR-MAINT-01	Maintainability	The parts are separate enough that swapping a sensor brand does not change the rest.	Sensor change stays local.	Code review
NFR-COMP-01	Compatibility	Works with HCMUT_SSO, DATACORE (read-only), and BKPay.	Matches each interface.	Integration test

8. Assumptions and open questions

Things we assumed, and would confirm with the university if this were real:

- We treat HCMUT_SSO as a CAS server (the login page says it runs Apereo CAS), so we log in by redirecting to it, getting a ticket back, and validating that ticket on our server. If the graders would rather we model it as a generic OAuth/OIDC provider, that is a small change.
- DATACORE is read-only. We create the local user on first login from what the SSO gives us and fill in the rest from DATACORE, treating our copy as a cache. We never write back to DATACORE.
- BKPay does not give us an API we can call for parking, so we stub it. We designed the flow the way the common Vietnamese gateways work (redirect to pay, then a server-to-server callback tells us it succeeded), and left a real BKPay hookup as future work.
- The main vehicle is the motorbike and the main lane is barrier-free. Barriers are only for the car lot.
- We count occupancy by adding one on entry and subtracting one on exit, and reset at night to fix drift.

Open questions we could not answer on our own:

- Which fields does the SSO actually return to a service like ours (name, email, and importantly whether the person is a student, lecturer, or staff)? That decides how much we need from DATACORE.
- Does DATACORE give us a query API or only a nightly export? Does it hold the vehicle/plate registration?
- Is there any BKPay path for small payments, or do we have to use the "service fee" channel? Would a prepaid wallet be acceptable to finance?
- Would the university register our app as a CAS service and give it a service URL?

9. Glossary

- ANPR: the camera reading of the license plate.
- CAS: the ticket-based login protocol HCMUT_SSO uses.
- BKNetID / MSSV: the HCMUT account / student ID used to log in.
- BKPay: HCMUT's payment platform.
- DATACORE: the read-only source of student and vehicle data.
- Role: what a user can do in the app (end user, operator, admin).
- MQTT: the lightweight messaging the sensors use to send updates to the backend.
- Session: one park, from entry to exit. This is what we bill.
- Unknown slot: a slot whose sensor has not reported for too long, so we do not trust its state.